

University of South Florida

CDA 4205L Computer Architecture Lab

Lab Report

Semester: Fall 2023		
Experiment	Number:	2
	Date:	09/07/2023
Lab	Section:	001
	Lab TA:	Benjamin Kreiger
Report	Due Date:	09/14/2023
Group (Lab3 onwards)	Member #1 name:	Diya Jain
	Member #2 name:	

ABSTRACT:

The purpose of this lab is to help us better understand abstraction in computer architecture and how different hardware designs can impact power, performance, and area.

INTRODUCTION:

In this lab, we were supposed to work with the multiplication operand. The operation takes two operands, multiplies them together, and generates a product. There are two microarchitectures that exist which makes the multiplication happen. The first one is a just a dedicated multiplication circuit and the second is repeated addition.

METHOD:

Software Tools Used:

1. RARS (Runtime Simulator)
2. RISC-V

Files:

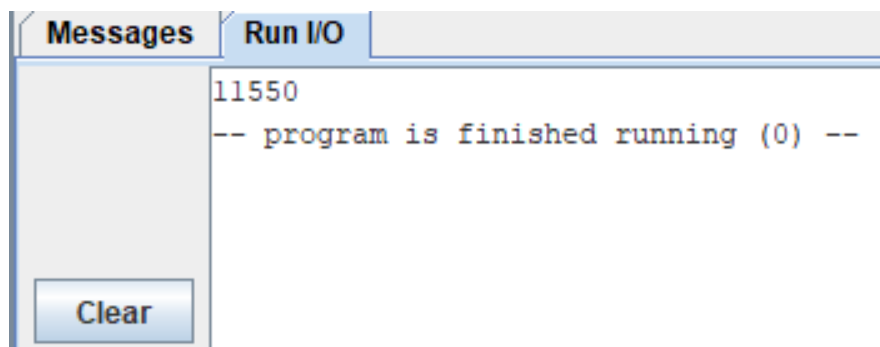
1. *uArch_1* – The file that has the multiplier circuit.
2. *uArch_2* – The file that has the repeated addition method.

RESULTS:

There was total 7 tasks that we were supposed to perform.

T1: Implement the function that executes the multiplier circuit writing the 'mul' operation on file uArch_1.asm

```
uArch_1.asm
3
4 .globl __main
5
6 .data
7
8
9 .text
10
11 __main:
12     li a7, 1          # pre-load th
13
14     li a0, 550         # load the fi
15     li a1, 21          # load the se
16     jal _multiply      # call the "_multiply
17
18     ecall              # _multiply r
19                       # so we can s
20
21     li a7, 10          # load the exit ecall
22     ecall              # exit the pr
23
24 # multiply function
25 # inputs: a0, a1
26 # output: a0 = a0 * a1
27 _multiply:
28     #---- your code here
29     mul a0,a0,a1
30
31     #---- end of your code
32     jr ra
33
```

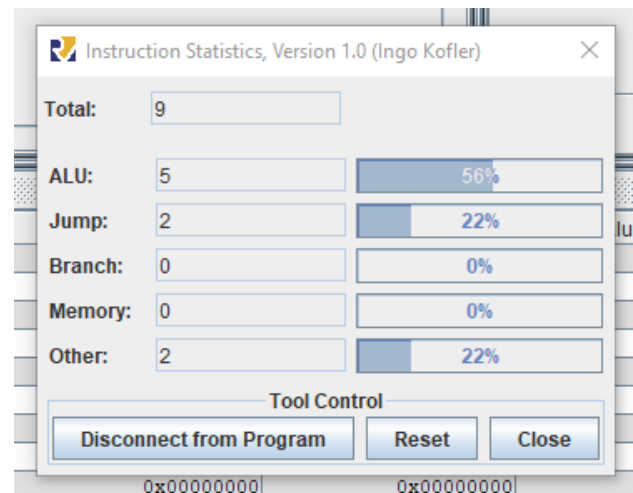


T2: How many total instructions were there? How many of each type?

The total number of instructions are 9.

The number of instructions for each type is:

1. ALU: 5
2. Jump: 2
3. Branch: 0
4. Memory: 0
5. Other: 2



T3: Implement the function that executes the repeated addition function on the file uArch_2.asm.

```
uArch_1.asm  uArch_2.asm
3
4 .globl __main
5
6 .data
7
8
9 .text
10
11 __main:
12     li a7, 1          # pre-load the Print
13     li a0, 550        # load the first op
14     li a1, 21         # load the second op
15     li a2, 0
16     jal _multiply     # call the "_multiply" func
17
18     ecall             # _multiply returns
19                     # so we can simply
20
21     li a7, 10         # load the exit ecall
22     ecall             # exit the program
23
24 # multiply function
25 # inputs: a0, a1
26 # output: a0 = a0 * a1
27 _multiply:
28     #---- your code here
29     beqz a1, _done
30     add a2, a2, a0
31     addi a1, a1, -1
32     j _multiply
33 _done:
34     addi a0, a2, 0
35     #---- end of your code
36     jr ra
37
```

Messages Run I/O

```
11550
-- program is finished running (0) --
```

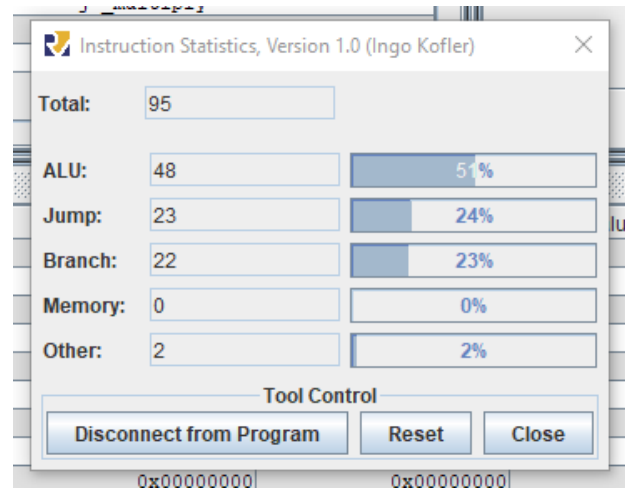
Clear

T4: How many total instructions were there? How many of each type?

The total number of instructions are 95.

The number of instructions for each type is:

1. ALU: 48
2. Jump: 23
3. Branch: 22
4. Memory: 0
5. Other: 2



T5: Assume the implementation of uArch_1 has an area of 8 μm^2 the implementation of uArch_2 has an area of 7 μm^2 . What is the area overhead for the multiplier? Report your result both as a raw value as well as a percent overhead.

$$\begin{aligned} R_1 &= 8 \mu\text{m}^2 \\ R_0 &= 7 \mu\text{m}^2 \\ \text{Raw Overhead} &= R_1 - R_0 \\ &= 8 - 7 \\ &= 1 \mu\text{m}^2 \\ \text{Percent Overhead} &= \frac{R_1 - R_0}{R_0} \times 100 \\ &= \frac{8 - 7}{7} \times 100 \\ &= \frac{1}{7} \times 100 = 14.28\% \end{aligned}$$

T6: Assume the multiplication itself consumes 500 pJ of energy, whereas other ALU instructions (those used for arithmetic such as add or addi excluding branch/comparison/etc.) consume 4 pJ each. If performance and area are not considered, at what point is it better to use uArch_1 rather than uArch_2 in terms of energy?

From the below picture we can see that the using uArch_1 is better than uArch_2 because the amount of energy used in uArch_1.asm is less than uArch_2.asm.

$$\begin{aligned} \text{Total Energy} &= \sum (E_i * N_i) && \text{--- ① For uArch_1.asm} \\ &= 4 * 5 \\ &= 20 \text{ pJ} \end{aligned}$$
$$\begin{aligned} \text{Total Energy} &= \sum (E_i * N_i) && \text{--- ② For uArch_2.asm} \\ &= 4 * 48 \\ &= 192 \text{ pJ} \end{aligned}$$

T7: Compute the EDP for uArch_1 and uArch_2.

$$\begin{aligned} \text{EDP} &= E * D && \text{--- For uArch_1.asm} \\ &= (4 * 5) * (500 * 5) \\ &= 50,000 \end{aligned}$$
$$\begin{aligned} \text{EDP} &= E * D && \text{--- For uArch_2.asm} \\ &= (4 * 47) * (500 * 47) \\ &= 188 * 23500 \\ &= 441800 \end{aligned}$$

DISCUSSION:

This lab was a more challenging compared to the first lab. The first two tasks were simple but doing multiplication with repeated addition had some challenges. We found that the energy consumed depends on the design. And depending on the numbers that are input, one or the other implementation (multiply built-in or for loop with repeated addition) is more efficient- not just always the shortest code! In future work, we will now keep in mind that it is not just about the results of the code but how the code itself is written because it affects the performance and power of the hardware. We did not notice any shortcomings in our work.

CONCLUSION:

In conclusion, this lab gives us a chance to understand abstraction in computer architecture more. Power, performance, and area are affected differently by different hardware designs. The tasks were to calculate and compare the results of the 2 implementations of multiplication to see the difference between the two. We noticed that the amount of energy consumed depends on the size of the numbers that are being multiplied. Though both ways of multiplying will result in the same number, it is important to understand that the way the assembly code itself is written will affect what happens in the actual process.